1. Central Thesis: The Intelligent AI Gateway

Our core strategic position is not to build a simple "LLM proxy." Proxies are lightweight intermediaries that forward requests, but they risk becoming a single point of failure and a latency bottleneck.[1]

Instead, we are building an **Intelligent AI Gateway**. A gateway is a comprehensive management layer that intercepts, observes, secures, and actively *enriches* the entire lifecycle of an LLM call.

The foundational principle of this architecture is to **eliminate negative impact** by ensuring our enrichment features do not introduce user-facing latency. We achieve this by building on an **asynchronous-first architecture**.[1] Simple, low-latency operations (like caching or key validation) happen in the synchronous path, while complex, high-latency operations (like memory consolidation or logging) are handled asynchronously "from the sidelines".[3]

This document first brainstorms all potential features this gateway could offer, then distills them into a prioritized Top 10 list to guide a low-cost, high-impact implementation.

2. Feature Brainstorm & Evaluation Matrix

This matrix contains a "sky's the limit" list of potential features, evaluated by their value, effort, and strategic fit.

- **Value:** (High, Medium, Low) - How much this feature solves a critical user problem (cost, latency, quality) or builds our unique moat.
- **Effort:** (Low, Medium, High) - The engineering cost and complexity to implement.

Feature	Description	Value	Effort	Category	Strategic Angle
--- Core Latency & Cost Features ---					
Semantic Caching	Intercepts a prompt, computes an embedding, and checks if a semantically similar answer already exists in the cache.[6]	**High**	Medium	Perf / Cost	A "must-have" gateway feature. Can cut 30%+ of API calls, providing immediate, tangible cost savings.[7]

Exact Match Caching	An L1 cache. If the prompt hash is identical, return the exact same response.[8]	Medium	**Low**	Perf / Cost	The simplest form of caching. Low hit rate in dynamic apps but cheap to build.
Context Compression	Automatically runs the prompt + context through a compression model (like LLMLingua-2 [9]) before sending it to the LLM.	**High**	Medium	Cost / Perf	**Our Moat.** We sell "cheaper, faster calls." Can reduce tokens by 26-54% [11] and boost RAG performance.[13]
KV Cache-Friendly Assembly	Intelligently re-orders the prompt to place static content (system prompt, memory) before dynamic content (user query) to maximize the provider's KV cache hit rate.[14]	Medium	**Low**	Performance	A "pro-tip" we build in by default. It's a free performance win for our users that demonstrates deep technical value.
--- Core Observability Features (Visibility) ---					
Asynchronous Tracing	The user's call passes through instantly, while logs (prompt, response,	**High**	Medium	**Foundation**	**This is the core low-latency enabler.** Our value is in

	latency, cost) are written to a database *asynchronously*.[1]				async observability, not a blocking inline service.
Cost & Token Dashboards	A UI that tracks token usage and cost, filterable by user, model, or time.	**High**	**Low**	Observability	Users *need* this. It's the primary way to prove ROI. Built on top of the async trace log.[15]
Latency Dashboards	Tracks p50, p95, and p99 latency for all calls, helping users identify slow models or prompts.[15]	**High**	**Low**	Observability	Paired with cost, this is the other half of the performance dashboard.
A/B Testing Framework	**(User Request)** Allows a user to define two prompts ("A" and "B") for the same query. The gateway routes 50% of traffic to each and logs the results for comparison.	**High**	Medium	Observability	The gateway is the *perfect* place to implement this. A huge win for prompt engineers.
Prompt Regression Testing	Logs and "snapshots" a known-good response, then alerts the user if a new model version or prompt change causes a	Medium	Medium	Observability	Solves a major pain point: "My prompts broke after the last OpenAI update."

	significant deviation.[16]				
User Feedback Endpoint	Provides an API endpoint (/feedback) so the user's app can send "thumbs up/down" signals, which we tie directly to the trace log.[17]	Medium	**Low**	Observability	Closes the loop. Allows us to measure qualitative metrics, not just quantitative ones.[19]
--- Core Gateway & Security Features (Control) ---					
Unified API	A single API endpoint that can route requests to different providers (OpenAI, Anthropic, Google) based on a header or prompt field.	**High**	Medium	Routing	**This is the "Gateway" part.**[20] It abstracts the provider, preventing vendor lock-in for our users.
Virtual Key Management	Users store their main provider API key with us once. We issue them *our* virtual keys, which they use in their app.	**High**	**Low**	Security	Solves a huge security problem (not exposing the main key). Allows us to implement rate limiting and budgeting.
Rate Limiting & Budgeting	**(User Request)** Per-virtual-key rate limiting	**High**	**Low**	Security / Cost	This is a non-negotiable for enterprise customers.

	and hard budget caps ("stop all calls from this key after $50").				Easy to build on virtual keys.
PII Detection & Masking	Scans prompts and responses for PII (names, emails, SSNs) and can be configured to either redact it or block the request.[1]	Medium	Medium	Security	Essential for compliance in healthcare, finance.
Audit Trails	A detailed, immutable log of who made what call at what time, with what prompt.	Medium	**Low**	Security	A simple logging feature, but critical for enterprise-grade governance.
--- The "Cognitive" Memory Moat (Enrichment) ---					
Asynchronous Consolidation	**The Core Patentable Idea.** An async background worker [3] uses an LLM to read the raw log, extract structured facts, and *write* them to the memory store.[4]	**High**	**High**	**Memory Moat**	This is the *entire business*. It's how we build the cognitive layer without adding latency.[3]
Factual Memory (K/V Store)	Injects simple, structured facts (e.g., dislikes: "coffee") into	**High**	**Low**	**Memory Moat**	Easy to build (a simple K/V store or SQL table [26]) and *perfect* for the

	the prompt. Solves the "goldfish memory" problem.[24]				first POC to show value beyond RAG.[26]
Episodic Memory (Graph)	Injects *relationships* (e.g., "User *discussed* Project X *with* Alice last week"). The "Star Wars birthday" feature.[27]	**High**	**High**	**Memory Moat**	The long-term vision. This is true cognitive, "human-like" memory.[29] Uses a graph DB.[34]
Cross-Platform Neutrality	The memory store is unified. A fact learned via an OpenAI call is injected into a future Anthropic call.	**High**	Medium	**Strategic Moat**	**This is our core business defense.** Incumbents (AWS, SNOW) are siloed.[4] We solve "memory fragmentation".[37]
--- Advanced Enrichment & Routing (The "Go Crazy" Features) ---					
Temporal Injection	**(User Request)** Gateway automatically injects context: "Today is Friday, Q4, 4:50 PM."	Medium	**Low**	Enrichment	A "wow" feature that's trivial to implement. It's a simple pre-hook that enriches the prompt.
Sentiment-Based Routing	Analyze user sentiment. If highly negative, automatically	Medium	Medium	Adv. Routing	A smart, adaptive feature. Shows the gateway is "intelligent,"

	route to the best/most-empathetic model or flag for human review.				not just a dumb pipe.
Error-Driven Intervention	**(User Request)** Observes a pattern (e.g., 3+ failed API calls from a user). Proactively injects a new instruction: "User is struggling. Offer to simplify the task."	Medium	Medium	Enrichment	This is a truly *agentic* gateway. It observes patterns and *acts* to improve the outcome.

3. The Top 10: A Prioritized Feature Roadmap

This is the distilled, high-value, low-effort roadmap designed for a team with a limited budget. It builds a foundation of immediate, tangible value (cost, security, latency) that hooks users, and then layers on the unique, "sticky" memory features that create our moat.

1. **Asynchronous Observability & Logging (Effort: Medium-Low)**
    - **Value:** This is the non-negotiable foundation. It's the "slide-in-place" part. All other features (cost tracking, A/B testing, latency metrics) are built on this async data pipeline.[1] It ensures we **add zero latency** to the user's call.
2. **Virtual Key Management (Effort: Low)**
    - **Value:** An immediate security win for developers. They stop exposing their primary OpenAI key in their apps and get an abstraction layer that *only you* provide. This is the "lock" that all other features (budgeting, rate limiting) plug into.
3. **Cost & Latency Dashboards (Effort: Low)**
    - **Value:** This is your primary ROI dashboard. You build this by simply reading from the async log database (#1). It's the first thing you show potential customers: "See? Our gateway *proves* its value by showing you your exact costs and latencies".[15]
4. **Semantic Caching (Effort: Medium)**

- **Value:** The single biggest and most immediate cost-saver. This is your "magic" feature. "Turn on our gateway and instantly cut 30% of your LLM bill".[7] It's a massive, tangible win that hooks users immediately.

5. **Unified API & Smart Routing (Effort: Medium)**
   - **Value:** This is your "vendor-agnostic" moat. It's the first step to becoming the *neutral* cognitive layer.[37] Developers love this flexibility. You can start by routing to just OpenAI and Anthropic, proving the concept.
6. **Cost-Based Budgeting & Throttling (Effort: Low)**
   - **Value:** Directly addresses your request. It's a simple check against the data from (#3) and the keys from (#2). This is a critical feature for any business trying to control runaway AI spend.
7. **Factual Memory (K/V Store) (Effort: Low-to-Medium)**
   - **Value:** This is your *first* "cognitive" feature. It's easy to build (a simple, fast Redis or SQL lookup [26]) and perfectly demonstrates the "I don't like coffee" problem that RAG alone can't solve.[26] This is your perfect MVP for the memory moat.
8. **Temporal Prompt Injection (Effort: Low)**
   - **Value:** Directly addresses your request. This is a high-impact "wow" feature with almost zero effort. A simple pre-processing hook that injects Context: Today is Friday, November 8, 2025. It is 4:52 PM. makes you look incredibly smart and demonstrates the "enrichment" value.
9. **Context Compression (via LLMLingua-2) (Effort: Medium)**
   - **Value:** This is your second "magic" feature and a core part of your moat. You integrate an open-source model like LLMLingua-2 [9] and offer it as a simple checkbox: "Enable Compression." You can now *measurably* reduce your users' token costs and latency on *every single call* that isn't a cache hit.[11]
10. **A/B Testing Framework (Effort: Medium)**
    - **Value:** Directly addresses your request. This is a power-user feature that your open-source community will love. It shows you're building a tool for *developers* and prompt engineers, not just a simple utility. It's the foundation for all prompt optimization.

Works cited

1. Should you use an LLM Proxy to Build your Application? - Langfuse ..., accessed on November 7, 2025, https://langfuse.com/blog/2024-09-langfuse-proxy
2. What is Agentic RAG? | IBM, accessed on November 7, 2025, https://www.ibm.com/think/topics/agentic-rag
3. Reactive Transformer (RxT): Fixing the Memory Problem in ..., accessed on November 7, 2025, https://huggingface.co/blog/AdamF92/reactive-transformer-intro
4. Amazon Bedrock AgentCore Memory: Building context-aware ..., accessed on November 7, 2025, https://aws.amazon.com/blogs/machine-learning/amazon-bedrock-agentcore-m

emory-building-context-aware-agents/
5. Mem0: Building Production-Ready AI Agents with Scalable Long ..., accessed on November 7, 2025, https://arxiv.org/pdf/2504.19413
6. Common Solutions to Latency Issues in LLM Applications | by fernandooo - Medium, accessed on November 7, 2025, https://medium.com/@mancity.kevindb/common-solutions-to-latency-issues-in-llm-applications-d58b8cf4be17
7. Using Redis for real-time RAG goes beyond a Vector Database, accessed on November 7, 2025, https://redis.io/blog/using-redis-for-real-time-rag-goes-beyond-a-vector-database/
8. Ultimate Guide to LLM Caching for Low-Latency AI - Ghost, accessed on November 7, 2025, https://latitude-blog.ghost.io/blog/ultimate-guide-to-llm-caching-for-low-latency-ai/
9. Learn Compression Target via Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression - LLMLingua-2, accessed on November 7, 2025, https://llmlingua.com/llmlingua2.html
10. microsoft/LLMLingua: [EMNLP'23, ACL'24] To speed up LLMs' inference and enhance LLM's perceive of key information, compress the prompt and KV-Cache, which achieves up to 20x compression with minimal performance loss. - GitHub, accessed on November 7, 2025, https://github.com/microsoft/LLMLingua
11. ACON: Optimizing Context Compression for Long-horizon LLM Agents - Emergent Mind, accessed on November 7, 2025, https://www.emergentmind.com/articles/2510.00615
12. Acon: Optimizing Context Compression for Long-horizon LLM Agents - arXiv, accessed on November 7, 2025, https://arxiv.org/html/2510.00615v1
13. LLMLingua Series - Microsoft Research: Longllmlingua, accessed on November 7, 2025, https://www.microsoft.com/en-us/research/project/llmlingua/longllmlingua/
14. Latency optimization - OpenAI API, accessed on November 7, 2025, https://platform.openai.com/docs/guides/latency-optimization
15. AI Agent Evaluation from Prompts to Metrics - IBM, accessed on November 7, 2025, https://www.ibm.com/think/tutorials/ai-agent-evaluation
16. An Empirical Study of Testing Practices in Open Source AI Agent Frameworks and Agentic Applications - arXiv, accessed on November 7, 2025, https://arxiv.org/html/2509.19185v1
17. A Complete Guide on AI Agent PoC: From Idea to Execution - Biz4Group LLC, accessed on November 7, 2025, https://www.biz4group.com/blog/ai-agent-poc
18. AI Metrics that Matter: A Guide to Assessing Generative AI Quality - Encord, accessed on November 7, 2025, https://encord.com/blog/generative-ai-metrics/
19. A complete guide to RAG evaluation: metrics, testing and best practices - Evidently AI, accessed on November 7, 2025, https://www.evidentlyai.com/llm-guide/rag-evaluation
20. How should Enterprises evaluate LLM Gateway for Scale? - TrueFoundry, accessed on November 7, 2025,

https://www.truefoundry.com/blog/how-should-enterprises-evaluate-llm-gateway-for-scale
21. API Gateway vs API Proxy: Understanding The Differences - Kong Inc., accessed on November 7, 2025, https://konghq.com/blog/engineering/api-gateway-vs-api-proxy-understanding-the-differences
22. LLM proxy vs AI gateway: what's the difference and which one do you need? - Portkey, accessed on November 7, 2025, https://portkey.ai/blog/llm-proxy-vs-ai-gateway/
23. US20240414108A1.pdf
24. Everyone's trying vectors and graphs for AI memory. We went back to SQL. : r/AI_Agents - Reddit, accessed on November 7, 2025, https://www.reddit.com/r/AI_Agents/comments/1nkx0bz/everyones_trying_vectors_and_graphs_for_ai_memory/
25. Everyone's trying vectors and graphs for AI memory. We went back ..., accessed on November 7, 2025, https://www.reddit.com/r/AI_Agents/comments/1nkx0bz/everyone_trying_vectors_and_graphs_for_ai_memory/
26. RAG vs Memory for AI Agents: What's the Difference - DEV Community, accessed on November 7, 2025, https://dev.to/bobur/rag-vs-memory-for-ai-agents-whats-the-difference-2ad0
27. RAG is not Agent Memory | Letta, accessed on November 7, 2025, https://www.letta.com/blog/rag-vs-agent-memory
28. RAG Vs Memory in AI Agent - by Naresh Kancharla - Medium, accessed on November 7, 2025, https://medium.com/@naresh.kancharla/rag-vs-memory-in-ai-agent-95c996ff1ad7
29. A Machine with Short-Term, Episodic, and Semantic Memory Systems, accessed on November 7, 2025, https://ojs.aaai.org/index.php/AAAI/article/view/25075/24847
30. Episodic Memory in AI Agents - GeeksforGeeks, accessed on November 7, 2025, https://www.geeksforgeeks.org/artificial-intelligence/episodic-memory-in-ai-agents/
31. Episodic memory in ai agents poses risks that should be studied and mitigated - arXiv, accessed on November 7, 2025, https://arxiv.org/html/2501.11739v1
32. Differential reorganization of episodic and semantic memory systems in epilepsy-related mesiotemporal pathology - PMC - PubMed Central, accessed on November 7, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC11531848/
33. The shared and unique neural correlates of personal semantic, general semantic, and episodic memory | eLife, accessed on November 7, 2025, https://elifesciences.org/articles/83645
34. Build AI Agents with Memory: LangChain + FalkorDB, accessed on November 7, 2025, https://www.falkordb.com/blog/building-ai-agents-with-memory-langchain/
35. Building a Memory-Aware AI with Knowledge Graphs: A Technical ..., accessed on

November 7, 2025, https://medium.com/@mailtoksingh08/building-a-memory-aware-ai-with-knowledge-graphs-a-technical-deep-dive-b9908b3edf94
36. AI Agent Practical Implementation: Lessons from th... - ServiceNow ..., accessed on November 7, 2025, https://www.servicenow.com/community/now-assist-articles/ai-agent-practical-implementation-lessons-from-the-field/ta-p/3339826
37. LLM Memory Types & AI Memory Limitations - Plurality Network, accessed on November 7, 2025, https://plurality.network/blogs/ai-memory-limitations-and-llm-memory-types/
38. Redis Use Cases in LLM Applications | Upstash Blog, accessed on November 7, 2025, https://upstash.com/blog/redis-in-llms